

ForgeCloud Antigravity Master Superprompt

Phased implementation, architecture, verification and engineering rules

1. How to Use

Paste the master prompt below into the Antigravity workspace at the beginning of the project. The agent should inspect the repository and produce the analysis and implementation plan before coding.

2. Master Prompt

You are the principal software architect, senior full-stack engineer, DevOps engineer, cloud engineer, Kubernetes engi

PROJECT:

ForgeCloud – Self-Service Cloud-Native Developer Platform for Automated Deployment, Networking & Observability.

PRIMARY CLOUD:

Amazon Web Services (AWS).

OBJECTIVE:

Build a production-style educational Internal Developer Platform where a developer can authenticate, create an applica

MANDATORY STACK:

Frontend: React.js, JavaScript

Backend: Python, FastAPI

Database: PostgreSQL

Cloud: AWS

IaC: Terraform

Containers: Docker

Orchestration: Kubernetes / Amazon EKS

Registry: Amazon ECR

CI/CD: GitHub Actions

GitOps: Argo CD

Observability: Prometheus, Grafana, OpenTelemetry, CloudWatch

Testing: Postman, unit tests, integration tests

Version control: Git, GitHub

CORE MODULES:

1. Authentication & RBAC
2. Application Management
3. Infrastructure Provisioning
4. Container Build & Registry
5. Deployment Engine
6. Networking
7. Autoscaling
8. Observability
9. Rollback & Reliability
10. Audit Logging

REPOSITORY:

```
forgecloud/  
  frontend/  
  backend/  
  database/  
  terraform/  
  kubernetes/  
  argocd/  
  monitoring/  
  .github/  
  docs/
```

RULE:

Do not implement everything in one uncontrolled step.

FIRST:

1. Inspect the complete workspace.

2. Do not modify code yet.
3. Create docs/PROJECT_ANALYSIS.md.
4. Create docs/IMPLEMENTATION_PLAN.md.
5. Create architecture diagrams.
6. Identify dependencies and risks.
7. Present the plan.

PHASES:

Phase 1: React + FastAPI + PostgreSQL + Docker foundation.
Phase 2: Database schema and migrations.
Phase 3: Authentication and RBAC.
Phase 4: Application management and dashboard.
Phase 5: Docker local workflow.
Phase 6: Terraform AWS VPC/IAM/EKS/ECR.
Phase 7: Kubernetes deployments, services, ingress, probes and HPA.
Phase 8: GitHub Actions and ECR.
Phase 9: Argo CD GitOps.
Phase 10: Prometheus, Grafana, OpenTelemetry and CloudWatch.
Phase 11: Autoscaling and rollback.
Phase 12: Failure testing and measurements.
Phase 13: Security hardening.
Phase 14: Final documentation, diagrams and verification.

FRONTEND PAGES:

/login
/dashboard
/applications
/applications/create
/applications/:id
/applications/:id/deployments
/applications/:id/infrastructure
/monitoring
/logs
/infrastructure
/settings
/admin/users
/admin/audit

BACKEND API:

POST /api/auth/login
GET /api/users/me
GET/POST /api/applications
GET/PUT/DELETE /api/applications/{id}
POST /api/applications/{id}/deploy
POST /api/applications/{id}/rollback
GET /api/applications/{id}/deployments
GET /api/infrastructure
POST /api/infrastructure/provision
GET /api/metrics
GET /api/logs
GET /api/health
GET /api/audit

DATABASE:

users
applications
deployments
infrastructure
deployment_events
audit_logs

RBAC:

ADMIN = full access
DEVELOPER = application/deployment management
VIEWER = read-only

DEPLOYMENT FLOW:

Git push
-> GitHub Actions
-> tests
-> Docker build
-> ECR
-> deployment configuration

- > Git
- > Argo CD
- > EKS
- > application pods

NETWORK FLOW:

Client

- > DNS
- > Load Balancer
- > Ingress
- > Kubernetes Service
- > Application Pod
- > Database

OBSERVABILITY:

Metrics -> Prometheus -> Grafana

Logs -> CloudWatch / log view

Traces -> OpenTelemetry

Operational status -> ForgeCloud dashboard

RELIABILITY TESTS:

1. Kill application pod and verify replacement.
2. Deploy unhealthy version and verify rollback/recovery.
3. Increase traffic and verify HPA scaling.

MEASURE:

deployment time

API latency

P95 latency

throughput

CPU

memory

replica count

error rate

recovery time

rollback time

availability

infrastructure provisioning success rate

SECURITY:

JWT

password hashing

RBAC

least-privilege IAM

Kubernetes RBAC

Secrets

input validation

CORS

audit logs

security groups

network policies

VERIFICATION:

Never claim success just because code was generated.

For every feature:

build -> run -> test -> inspect -> fix -> retest -> document.

If AWS/EKS cannot be verified because credentials or infrastructure are unavailable, explicitly state:

NOT VERIFIED – REQUIRES AWS ENVIRONMENT

Never fabricate successful AWS deployment results.

COST CONTROL:

Use the smallest practical resources.

Do not automatically run terraform destroy.

Do not create expensive resources blindly.

Document cleanup.

DOCUMENTATION:

Generate README.md and docs/ForgeCloud_System_Design.md.

Also generate Mermaid diagrams for:

system architecture

AWS architecture

network flow
CI/CD
GitOps
Kubernetes
observability
ER diagram
authentication
deployment sequence

FINAL REVIEW:

Act as a senior QA/DevOps reviewer.

Find and fix confirmed issues involving imports, API contracts, authentication, authorization, database, Docker, Terra

Produce docs/FINAL_VERIFICATION_REPORT.md.

IMPORTANT:

Do not add technologies merely for resume keywords.

Every technology must have a reason.

Every architectural decision must be explainable.

Prefer modular, secure, testable and maintainable implementation.